
Security of Computer Systems

Project Report

Authors:
Adam Klamrowski 198199

Version: 1.4

Versions

Version	Date	Description of changes
1.0	07.04.2026	Creation of the document and initial description of the TTP-based environment.
1.1	15.05.2026	Added registration, initiation, routing, proxy configuration and basic client-server message flow.
1.2	23.05.2026	Added TTP initiation, client and server internal logic and container network assumptions.
1.3	30.05.2026	Added authentication, certificate checking, certificate persistence and secure message interfaces.
1.4	31.05.2026	Final version: added protocol refactoring, identity/crypto separation and generated documentation with godoc, gomarkdoc and Doxygen.

1. Project – control term

1.1 Description

The goal of the control-term stage was to define and prepare the architecture for a secure distributed environment based on a Trusted Third Party (TTP). The project was structured as a set of independent Go services: a client application, a target server and a TTP service. Each component was separated into its own command entry point and internal application modules, which made the communication flow and security responsibilities clear.

At this stage, the project scope included preparing the application structure, defining the protocol messages and selecting the cryptographic mechanisms used later in the implementation. The planned solution was based on local application identities generated in the shared identity package. Each entity uses a public identifier, an RSA key pair for authentication, an RSA key pair for encryption and a certificate received from the TTP. The protocol uses X.509 certificates signed by the TTP, digital signatures for identity verification, RSA-OAEP with SHA-256 for protected key transport and AES-GCM for encrypted communication after successful authentication.

The target architecture separated the system into three trust domains. The client is responsible for initiating authentication and sending encrypted messages. The server verifies requests and forwards authentication data to the TTP. The TTP acts as the central authority responsible for registration, certificate generation, certificate validation and session key distribution. This separation reduced shared state between applications and made the trust assumptions easier to verify during the final implementation.

1.2 Results

The following results were prepared for the control term:

- The system architecture was divided into three independent Go applications: the client, the target server and the Trusted Third Party server.
- The communication sequence was defined for identity registration, certificate issuing, authentication, server-side verification and session-key distribution.
- The initial project structure was prepared using separate command entry points and shared internal packages for identity management, cryptographic operations, protocol messages and service logic.
- The security model was based on registered public keys, X.509 certificate validation, digital signatures and rejection of identities or keys that do not match the trusted data maintained by the TTP.

1.3 Summary

The control-term stage delivered the protocol design, the planned component architecture and the initial implementation direction for the secure distributed system. The remaining work for the final term was to complete the full authentication flow between the client, the target server and the TTP server.

The final implementation tasks included generating and verifying X.509 certificates, validating registered identities, delivering the session key to the communicating parties and implementing encrypted data exchange using the established secure channel. The project also required completing the source-code documentation and describing the implemented cryptographic mechanisms, protocol flow and application structure in the final report.

2. Project – Final term

2.1 Description

The main objective of the final project was to design, implement and analyze the security of an IT environment based on an authentication and key-distribution protocol with a Trusted Third Party. The final system was implemented as a set of independent Go applications communicating with each other through clearly separated protocol messages.

The system contains three cryptographically and communicationally separated nodes:

- Client Application: the application responsible for initiating the communication, storing its own identity, requesting authentication and using the received session key for encrypted communication.
- Target Server Application: the protected service that receives client requests, verifies authentication data and communicates with the TTP during the authentication process.
- Trusted Third Party Server: the centralized trusted authority responsible for identity registration, certificate generation, certificate validation and session-key distribution.

The implemented protocol is divided into three consecutive stages:

- 1.Registration and certification of entities: the client and the target server generate their own identities and cryptographic key pairs. Each entity owns a public identifier, an RSA key pair used for authentication and an RSA key pair used for encryption. Public identity data is registered in the TTP, which then issues an X.509 certificate signed by the trusted authority.
- 2.Mutual authentication and session-key distribution: when the client attempts to access the protected server, the authentication protocol is initiated. The server verifies the client request and forwards the required authentication data to the TTP. The TTP validates certificates, signatures and registered public keys. After successful validation, the TTP generates a symmetric session key and sends it separately to the client and the server, encrypted with RSA-OAEP using SHA-256 and the public encryption keys of both parties.
- 3.Direct communication in a secure channel: after receiving and decrypting the session key, the client and the target server communicate directly without involving the TTP in further application messages. The transferred application data is protected with symmetric encryption using AES-GCM. Each encrypted

message uses a unique random nonce, which provides confidentiality and integrity protection for the exchanged data.

2.2 Code Description

Mitigation of Man-in-the-Middle Attacks and Forged Certificates

One of the most critical threats in distributed key exchange systems is a situation in which an attacker intercepts network traffic and replaces the original public key or certificate with their own. The implemented TTP application mitigates this threat during the authentication stage by comparing the certificate received in the request with the historical registration data stored by the Trusted Third Party.

The authentication process verifies whether the declared server and client are registered entities, checks their assigned roles, validates their X.509 certificates and verifies digital signatures. A crucial part of this mechanism is the comparison between the public key contained in the submitted certificate and the public key stored during the registration phase. If the keys do not match, the authentication process is rejected.

```
func (s *Service) Authenticate(req protocol.AuthenticateRequest) (protocol.AuthenticateResponse, error) {
    ttpEncPrivateKey, err := identity.LoadPrivateKey(filepath.Join(s.baseDir, "enc.key"))
    if err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("load ttp enc private key: %w", err)
    }

    serverEntity, exists := s.entries[req.ServerID]
    if !exists {
        return protocol.AuthenticateResponse{}, fmt.Errorf("server not registered")
    }

    if serverEntity.Role != string(protocol.EntityRoleServer) {
        return protocol.AuthenticateResponse{}, fmt.Errorf("entity is not server")
    }

    var ttpAuthPrivateKey *rsa.PrivateKey
    ttpAuthPrivateKey, err = identity.LoadPrivateKey(filepath.Join(s.baseDir, "auth.key"))
    if err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("load ttp auth private key: %w", err)
    }

    if err = ValidateCertificateBase64(
        req.ServerCertificate,
        serverEntity.ID,
        serverEntity.AuthPublicKey,
        &ttpAuthPrivateKey.PublicKey,
    ); err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("invalid server certificate: %w", err)
    }

    var serverAuthPublicKey *rsa.PublicKey
    serverAuthPublicKey, err = identity.ParsePublicKeyFromBase64(serverEntity.AuthPublicKey)
    if err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("parse server auth public key: %w", err)
    }

    if err = VerifySignatureBase64([]byte(req.ServerID), req.ServerSignature, serverAuthPublicKey); err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("invalid server signature: %w", err)
    }

    var clientPayloadBytes []byte
    clientPayloadBytes, err = DecryptLargePayloadWithPrivateKeyBase64(
        req.ClientEncryptedPayload,
        ttpEncPrivateKey,
    )
    if err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("decrypt client payload: %w", err)
    }

    var clientPayload protocol.AuthenticateClientPayload
    if err = json.Unmarshal(clientPayloadBytes, &clientPayload); err != nil {
        return protocol.AuthenticateResponse{}, fmt.Errorf("decode client payload: %w", err)
    }
}
```

```

var clientEntity RegisteredEntity
clientEntity, exists = s.entries[clientPayload.ClientID]
if !exists {
    return protocol.AuthenticateResponse{}, fmt.Errorf("client not registered")
}

if clientEntity.Role != string(protocol.EntityRoleClient) {
    return protocol.AuthenticateResponse{}, fmt.Errorf("entity is not client")
}

if err = ValidateCertificateBase64(
    clientPayload.ClientCertificate,
    clientEntity.ID,
    clientEntity.AuthPublicKey,
    &tpAuthPrivateKey.PublicKey,
); err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("invalid client certificate: %w", err)
}

var clientAuthPublicKey *rsa.PublicKey
clientAuthPublicKey, err = identity.ParsePublicKeyFromBase64(clientEntity.AuthPublicKey)
if err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("parse client auth public key: %w", err)
}

if err = VerifySignatureBase64([]byte(clientPayload.ClientID), clientPayload.ClientSignature, clientAuthPublicKey); err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("invalid client signature: %w", err)
}

var sessionKey []byte
sessionKey, err = identity.GenerateRandomBytes(32)
if err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("generate session key: %w", err)
}

var clientEncPublicKey *rsa.PublicKey
clientEncPublicKey, err = identity.ParsePublicKeyFromBase64(clientEntity.EncPublicKey)
if err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("parse client enc public key: %w", err)
}

var encryptedSessionKeyForClient string
encryptedSessionKeyForClient, err = identity.EncryptWithPublicKeyBase64(sessionKey, clientEncPublicKey)
if err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("encrypt session key for client: %w", err)
}

var serverEncPublicKey *rsa.PublicKey
serverEncPublicKey, err = identity.ParsePublicKeyFromBase64(serverEntity.EncPublicKey)
if err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("parse server enc public key: %w", err)
}

var encryptedSessionKeyForServer string
encryptedSessionKeyForServer, err = identity.EncryptWithPublicKeyBase64(sessionKey, serverEncPublicKey)
if err != nil {
    return protocol.AuthenticateResponse{}, fmt.Errorf("encrypt session key for server: %w", err)
}

return protocol.AuthenticateResponse{
    OK: true,
    EncryptedSessionKeyForClient: encryptedSessionKeyForClient,
    EncryptedSessionKeyForServer: encryptedSessionKeyForServer,
    Message: "authenticated",
}, nil
}

```

List. 1 - Authentication process and certificate validation in ttpservice.go.

The certificate validation function decodes the X.509 certificate, verifies its validity period, checks the declared subject identifier and compares the RSA public key from the certificate with the public key stored during registration. The comparison is performed by checking the RSA modulus and public exponent. If these values differ, the certificate is rejected as invalid or potentially forged.

```

func ValidateCertificateBase64(
    certificateBase64 string,

```

```

expectedSubjectID string,
expectedPublicKeyBase64 string,
issuerPublicKey *rsa.PublicKey,
) error {
    cert, err := ParseCertificateBase64(certificateBase64)
    if err != nil {
        return fmt.Errorf("parse certificate: %w", err)
    }

    now := time.Now()
    if now.Before(cert.NotBefore) || now.After(cert.NotAfter) {
        return fmt.Errorf("certificate expired or not yet valid")
    }

    if cert.Subject.CommonName != expectedSubjectID {
        return fmt.Errorf("invalid certificate subject: got=%s expected=%s",
            cert.Subject.CommonName,
            expectedSubjectID)
    }

    var expectedPublicKey *rsa.PublicKey
    expectedPublicKey, err = identity.ParsePublicKeyFromBase64(expectedPublicKeyBase64)
    if err != nil {
        return fmt.Errorf("parse expected public key: %w", err)
    }

    certPublicKey, ok := cert.PublicKey.(*rsa.PublicKey)
    if !ok {
        return fmt.Errorf("certificate does not contain RSA public key")
    }

    if certPublicKey.N.Cmp(expectedPublicKey.N) != 0 || certPublicKey.E != expectedPublicKey.E {
        return fmt.Errorf("certificate public key mismatch")
    }

    digest := sha256.Sum256(cert.RawTBSCertificate)

    if err = rsa.VerifyPKCS1v15(
        issuerPublicKey,
        crypto.SHA256,
        digest[:],
        cert.Signature,
    ); err != nil {
        return fmt.Errorf("invalid certificate signature: %w", err)
    }

    return nil
}

```

List. 2 - Public-key comparison and forged certificate detection in certificate.go.

Symmetric Data Exchange in a Secure Channel

After mutual trust has been established and the session key has been distributed by the TTP, the client and the server switch to symmetric cryptography. This approach provides better performance for further data exchange while maintaining confidentiality and integrity of the transmitted information.

The implemented server endpoint verifies whether an active session key exists, decrypts the incoming encrypted payload using the shared AES-256 session key, processes the client request and returns an encrypted response. In contrast to a basic CBC mode implementation, this project uses AES-GCM, which provides authenticated encryption and protects the message against unauthorized modification.

```

func (s *Server) handleMessage(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }

    sessionKey, err := identity.LoadSessionKey(s.baseDir)
    if err != nil {
        http.Error(w, "server is not authenticated - missing session key", http.StatusUnauthorized)
        return
    }
}

```

```

var request protocol.MessageRequest
if err = json.NewDecoder(r.Body).Decode(&request); err != nil {
    http.Error(w, "invalid request body", http.StatusBadRequest)
    return
}
defer func() {
    _ = r.Body.Close()
}()

if request.EncryptedBody == "" {
    http.Error(w, "encrypted body is required", http.StatusBadRequest)
    return
}

var plaintext []byte
plaintext, err = identity.DecryptWithSessionKeyBase64(request.EncryptedBody, sessionKey)
if err != nil {
    http.Error(w, "decrypt request: "+err.Error(), http.StatusUnauthorized)
    return
}

responseText := strings.ToUpper(string(plaintext))

var encryptedResponse string
encryptedResponse, err = identity.EncryptWithSessionKeyBase64([]byte(responseText), sessionKey)
if err != nil {
    http.Error(w, "encrypt response: "+err.Error(), http.StatusInternalServerError)
    return
}

response := protocol.MessageResponse{
    EncryptedBody: encryptedResponse,
}

_ = identity.DeleteSessionKey(s.baseDir)

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusOK)

if err = json.NewEncoder(w).Encode(response); err != nil {
    http.Error(w, "encode response", http.StatusInternalServerError)
    return
}
}

```

List. 3 - Symmetric encrypted data exchange in handlers.go.

The actual encryption and decryption operations are implemented in the shared identity package. The encryption function generates a random nonce, encrypts the plaintext using AES-GCM and serializes the resulting nonce and ciphertext into a Base64-encoded JSON payload.

```

func EncryptWithSessionKeyBase64(plaintext []byte, sessionKey []byte) (string, error) {
    if len(sessionKey) != 32 {
        return "", fmt.Errorf("invalid AES-256 session key length: %d", len(sessionKey))
    }

    block, err := aes.NewCipher(sessionKey)
    if err != nil {
        return "", err
    }

    var gcm cipher.AEAD
    gcm, err = cipher.NewGCM(block)
    if err != nil {
        return "", err
    }

    nonce := make([]byte, gcm.NonceSize())
    if _, err = rand.Read(nonce); err != nil {
        return "", err
    }

    ciphertext := gcm.Seal(nil, nonce, plaintext, nil)

    payload := HybridEncryptedPayload{
        Nonce:    base64.StdEncoding.EncodeToString(nonce),
        Ciphertext: base64.StdEncoding.EncodeToString(ciphertext),
    }
}

```



```

    }

    var payloadBytes []byte
    payloadBytes, err = json.Marshal(payload)
    if err != nil {
        return "", err
    }

    return base64.StdEncoding.EncodeToString(payloadBytes), nil
}

func DecryptWithSessionKeyBase64(encoded string, sessionKey []byte) ([]byte, error) {
    if len(sessionKey) != 32 {
        return nil, fmt.Errorf("invalid AES-256 session key length: %d", len(sessionKey))
    }

    payloadBytes, err := base64.StdEncoding.DecodeString(encoded)
    if err != nil {
        return nil, err
    }

    var payload HybridEncryptedPayload
    if err = json.Unmarshal(payloadBytes, &payload); err != nil {
        return nil, err
    }

    var nonce []byte
    nonce, err = base64.StdEncoding.DecodeString(payload.Nonce)
    if err != nil {
        return nil, err
    }

    var ciphertext []byte
    ciphertext, err = base64.StdEncoding.DecodeString(payload.Ciphertext)
    if err != nil {
        return nil, err
    }

    var block cipher.Block
    block, err = aes.NewCipher(sessionKey)
    if err != nil {
        return nil, err
    }

    var gcm cipher.AEAD
    gcm, err = cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }

    return gcm.Open(nil, nonce, ciphertext, nil)
}

```

List. 4 - AES-GCM encryption and decryption using the session key in encryption.go.

Asymmetric Encryption of the Identifier with the TTP Public Key

Before sending the entity identity through the network, both the client and the server encrypt their identifier with the public encryption key of the Trusted Third Party. This prevents the identifier from being transmitted in plain text and reduces the risk of exposing sensitive authentication data.

The following function uses RSA-OAEP with SHA-256 to encrypt the provided data with the public key of the receiver. The encrypted result is then encoded to Base64 so that it can be safely transmitted in JSON-based HTTP requests.

```

func EncryptWithPublicKeyBase64(data []byte, pub *rsa.PublicKey) (string, error) {
    hash := sha256.New()

    ciphertext, err := rsa.EncryptOAEP(hash, rand.Reader, pub, data, nil)
    if err != nil {
        return "", err
    }

    return base64.StdEncoding.EncodeToString(ciphertext), nil
}

```

List. 5 - Encrypting an identifier with the TTP public key in encryption.go.

The function is used during the bootstrap process of both the client and the server. After receiving the TTP public key, the application encrypts its local identifier and sends it to the TTP as part of the registration request.

```
func (a *App) Bootstrap() error {
    ttpPublicKey, err := a.ttpClient.Init()
    if err != nil {
        return fmt.Errorf("ttp init: %w", err)
    }

    err = identity.EnsureIdentity(a.config.BaseDir)
    if err != nil {
        return err
    }

    var data protocol.RegisterRequest
    data, err = identity.LoadRegistrationData(a.config.BaseDir)

    if err != nil {
        return fmt.Errorf("load registration data: %w", err)
    }

    var encryptedID string
    encryptedID, err = identity.EncryptWithPublicKeyBase64([]byte(data.EncryptedID), ttpPublicKey)
    if err != nil {
        return fmt.Errorf("encrypt id for ttp: %w", err)
    }

    var certificateBase64 string
    certificateBase64, err = a.ttpClient.Register(
        protocol.RegisterRequest{
            EncryptedID: encryptedID,
            EncPublicKey: data.EncPublicKey,
            AuthPublicKey: data.AuthPublicKey,
            Role:        protocol.EntityRoleClient,
        })

    if err != nil {
        return fmt.Errorf("ttp register: %w", err)
    }

    if err = identity.SaveCertificate(a.config.BaseDir, certificateBase64); err != nil {
        return fmt.Errorf("save certificate: %w", err)
    }

    return nil
}
```

List. 6 - Client registration with encrypted identifier in app.go.

Entity Registration and X.509 Certificate Generation

The registration method is located in the TTP service layer. It is responsible for decrypting the entity identifier, parsing the public authentication key submitted by the client or the server and issuing a digital X.509 certificate signed by the Trusted Third Party.

The generated certificate contains the entity identity in the common name field and includes key usage extensions required for digital signatures and key encipherment. This mechanism allows the TTP server to act as a trusted certificate authority within the implemented authentication protocol.

```
func (s *Service) Register(reg protocol.RegisterRequest) (protocol.RegisterResponse, error) {
    ttpEncPrivateKey, err := identity.LoadPrivateKey(filepath.Join(s.baseDir, "enc.key"))
    if err != nil {
        return protocol.RegisterResponse{}, err
    }

    var decryptedIDBytes []byte
    decryptedIDBytes, err = identity.DecryptWithPrivateKeyBase64(reg.EncryptedID, ttpEncPrivateKey)
    if err != nil {
        return protocol.RegisterResponse{}, err
    }
}
```

```

var userAuthPublicKey *rsa.PublicKey
userAuthPublicKey, err = identity.ParsePublicKeyFromBase64(reg.AuthPublicKey)
if err != nil {
    return protocol.RegisterResponse{}, err
}

var ttpAuthPrivateKey *rsa.PrivateKey
ttpAuthPrivateKey, err = identity.LoadPrivateKey(filepath.Join(s.baseDir, "auth.key"))
if err != nil {
    return protocol.RegisterResponse{}, err
}

var certificateBase64 string
certificateBase64, err = CreateCertificateBase64(
    string(decryptedIDBytes),
    userAuthPublicKey,
    ttpAuthPrivateKey,
)
if err != nil {
    return protocol.RegisterResponse{}, err
}

entityID := string(decryptedIDBytes)

if _, exists := s.entries[entityID]; exists {
    return protocol.RegisterResponse{}, fmt.Errorf("entity already registered")
}

s.entries[entityID] = RegisteredEntity{
    ID:      entityID,
    Role:    string(reg.Role),
    EncPublicKey: reg.EncPublicKey,
    AuthPublicKey: reg.AuthPublicKey,
    Certificate: certificateBase64,
}

return protocol.RegisterResponse{
    Certificate: certificateBase64,
}, nil
}

```

List. 7 - Entity registration and certificate issuance in ttpservice.go.

The certificate itself is created by the `CreateCertificateBase64` function. The function generates a random serial number, prepares an X.509 certificate template, assigns the entity identifier to the common name field and signs the certificate using the TTP authentication private key.

```

func CreateCertificateBase64(
    subjectID string,
    subjectPublicKey *rsa.PublicKey,
    issuerPrivateKey *rsa.PrivateKey,
) (string, error) {
    serialLimit := new(big.Int).Lsh(big.NewInt(1), 128)

    serialNumber, err := rand.Int(rand.Reader, serialLimit)
    if err != nil {
        return "", err
    }

    template := x509.Certificate{
        SerialNumber: serialNumber,

        SignatureAlgorithm: x509.SHA256WithRSA,

        Subject: pkix.Name{
            CommonName: subjectID,
        },

        NotBefore: time.Now(),
        NotAfter:  time.Now().Add(365 * 24 * time.Hour),

        KeyUsage: x509.KeyUsageDigitalSignature | x509.KeyUsageKeyEncipherment,

        ExtKeyUsage: []x509.ExtKeyUsage{
            x509.ExtKeyUsageClientAuth,
            x509.ExtKeyUsageServerAuth,
        },
    }

```

```

    BasicConstraintsValid: true,
    IsCA:                  false,
}

issuerTemplate := x509.Certificate{
    SerialNumber: big.NewInt(1),

    SignatureAlgorithm: x509.SHA256WithRSA,

    Subject: pkix.Name{
        CommonName: "SCS TTP",
    },

    NotBefore: time.Now(),
    NotAfter:  time.Now().Add(10 * 365 * 24 * time.Hour),

    KeyUsage: x509.KeyUsageCertSign | x509.KeyUsageDigitalSignature,

    BasicConstraintsValid: true,
    IsCA:                  true,
}

var certDER []byte
certDER, err = x509.CreateCertificate(
    rand.Reader,
    &template,
    &issuerTemplate,
    subjectPublicKey,
    issuerPrivateKey,
)
if err != nil {
    return "", err
}

return base64.StdEncoding.EncodeToString(certDER), nil
}

```

List. 8 - X.509 certificate generation and signing in certificate.go.

2.3 Description

The source code was documented using Doxygen, GoDoc and Go-Markdoc in order to provide consistent engineering documentation for the whole project. Since the backend components were implemented in Go, the source files were documented with idiomatic Go comments placed above packages, structures, interfaces, functions and methods. These comments were compatible with GoDoc and allowed the generated documentation to describe the purpose of individual modules, exported types and protocol-related functions.

Doxygen was additionally used to generate broader project documentation from the source tree. Its configuration processed the Trusted Third Party service, the target server, the client application and the shared protocol and identity packages recursively. This made it possible to inspect the overall structure of the implemented authentication and key exchange protocol, including packages, functions and dependencies between the main components.

Go-Markdoc was used to support Markdown-based technical documentation for the Go part of the project. It allowed the project documentation to be written in a readable form while still being connected with the structure and comments of the source code. As a result, the generated documentation combined source-level API descriptions with higher-level explanations of the implemented cryptographic mechanisms.

2.4 Results

From the security perspective, the most important result is the rejection of identities that attempt to authenticate using a public key different from the one stored during registration. This is the main protection against a Man-in-the-Middle scenario in which an attacker replaces a legitimate public key or certificate with their own cryptographic material.

After successful authentication, the TTP is no longer required for application traffic. The client and the target server communicate directly using the shared session key delivered during the authentication process. This reduces the role of the TTP to trust establishment and key distribution, while the actual business communication remains encrypted directly between the client and the target server.

The final version of the project implements a working TTP-based authentication and key-distribution protocol. The solution includes RSA key generation, X.509 certificate issuing, identity protection with RSA-OAEP SHA-256, validation of certificates against registered public keys, symmetric session-key delivery and encrypted data exchange between the client and the target server.

The final implementation achieved the following results:

Area	Implemented result	Security impact
Entity registration	Client and server register their public encryption and authentication keys in the TTP.	The TTP keeps a trusted historical reference for later validation.
Certificate issuing	The TTP signs X.509 certificates for registered entities.	Public keys are bound to declared identities.
MITM mitigation	Incoming certificate and public-key data is compared with the key stored during registration.	A substituted public key or forged certificate is rejected.
Session key delivery	A 256-bit symmetric key is generated and encrypted separately for both authenticated parties.	The session key is not transmitted in plaintext.
Secure service call	Client and server exchange encrypted payloads using AES-GCM and the shared session key.	Application data remains confidential and protected against unauthorized modification after authentication.
Documentation	Doxygen, GoDoc and Go-Markdoc documentation is generated for the Go source code and technical project description.	The implementation is easier to audit, understand and maintain.

3 Table 1 - Summary of implemented security mechanisms and their impact.

Overall, the project demonstrates how a Trusted Third Party can be used to establish trust between distributed services and how the risk of public-key substitution can be reduced by validating certificates and public keys against the trusted registration state maintained by the TTP. The implementation is supported by generated documentation, which improves maintainability and makes the security-critical packages and protocol functions easier to inspect.

3.1 Summary

The final version of the project implements a working TTP-based authentication and key-distribution protocol. The solution includes RSA key generation, X.509 certificate issuing, identity protection with RSA-OAEP SHA-256, validation of certificates against registered public keys, symmetric session-key distribution and encrypted data exchange between the client and the target server.

The project demonstrates how a Trusted Third Party can be used to establish trust between distributed services and how the risk of public-key substitution can be reduced by comparing certificates and public keys with the trusted registration state maintained by the TTP. The implementation uses RSA for asymmetric encryption and digital signatures, X.509 certificates for binding identities with public keys and AES-GCM for secure symmetric communication after authentication.

The implementation is supported by Doxygen, GoDoc and Go-Markdoc documentation. These tools improve maintainability of the project and make the security-critical packages, functions and protocol structures easier to inspect. As a result, both the low-level source code documentation and the higher-level technical description of the implemented protocol are available in a consistent form.

3. Literature

- [1] The Go Authors: The Go Programming Language Documentation, online access.
- [2] The Go Authors: Package `crypto/rsa`, Go standard library documentation, online access.
- [3] The Go Authors: Package `crypto/x509`, Go standard library documentation, online access.
- [4] The Go Authors: Package `crypto/aes` and `crypto/cipher`, Go standard library documentation, online access.
- [5] NIST: Special Publication 800-57 Part 1 Revision 5, Recommendation for Key Management, online access.
- [6] NIST: Special Publication 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC, online access.
- [7] RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile, online access.
- [8] RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2, online access.